



به نام خدا

دانشگاه تهران



دانشکده مهندسی برق و کامپیوتر

درس شبکه‌های عصبی و یادگیری عمیق

تمرین ششم

نام و نام خانوادگی	سید مانی میرشعبانی	پرسش ۱
شماره دانشجویی	810801080	
نام و نام خانوادگی	سید مانی میرشعبانی	پرسش ۲
شماره دانشجویی	810801080	

فهرست

قوانین	1
پرسش 1. یادگیری بدون نظارت و انتقال دامنه با استفاده از GAN	3
۱-۱. مقدمه	3
۱-۲. بخش نظری	3
۱-۲-۱. ناپایداری در GAN	3
۱-۲-۲. چرا مقاله فقط از نویز به عنوان ورودی مولد استفاده نمی‌کند؟	5
۱-۲-۳. اجزای مدل	5
۱-۲-۴. تابع هزینه Content similarity	7
۱-۲-۵. چرا فقط از تصاویر تولیدشده برای آموزش دسته‌بند استفاده نمی‌کنیم؟	8
۱-۲-۶. آیا این روش برای حوزه‌هایی با تفاوت معنایی (Semantic) قابل استفاده است؟	8
۱-۳. بخش عملی	9
۱-۳-۱. پیش‌پردازش داده‌ها	9
۱-۳-۲. مشاهده فاصله بین دو حوزه (Domain Gap)	10
۱-۳-۳. پیاده‌سازی معماری مدل	11
۱-۳-۴. توابع هزینه (Cost Functions)	16
۱-۳-۵. آموزش مدل	17
۱-۳-۶. نمایش نتایج	18
پرسش ۲ - بازسازی تصاویر پولیپ آندوسکپی با EndoVAE	21
۲-۱. پیش‌پردازش داده‌ها	21
۲-۲. معماری EndoVAE	22
۱. انکودر (Encoder)	22
۲. بازپارامترسازی (Reparameterization)	23
۳. دیکودر (Decoder)	23
۲-۳. توابع هزینه	25
۲-۴. آموزش مدل	27
۲-۵. تولید و بازسازی کیفی	29

29.....	۲-۵-۱. تولید (Generation)
31.....	۲-۵-۲. بازسازی (Reconstruction)
32.....	۲-۵. ارزیابی عددی (Numeric Evaluation)
34.....	۲-۷. تحلیل نتایج نهایی و بحث
34.....	۲-۷-۱. تحلیل کیفی
34.....	۲-۷-۲. تحلیل کمی
37.....	۲-۸. بخش امتیازی
37.....	۲-۸-۱. ساخت دیتاست
37.....	۲-۸-۲. ساخت و آموزش CNN
38.....	۲-۸-۳. ارزیابی شبکه‌ی CNN

شکل‌ها

- شکل 1: ساختار مدل مقاله.....6
- شکل 2: نمونه تصاویر دیتاست های MNIST و MNIST-M.....9
- شکل 3: ساختار دسته بند (Classifire) در مقاله.....10
- شکل 4: نمودار های خطا و دقت مدل روی داده های مبدا (MNIST) در حین آموزش.....11
- شکل 5: دقت مدل روی داده های تست MNIST و کل MNIST-M.....11
- شکل 6: نمودار ساختار کلی مدل.....12
- شکل 7: پارامتر های معماری مدل.....12
- شکل 8: جزئیات مدل به کار رفته برای کار ما (MNIST-M).....13
- شکل 9: ساختار مولد.....14
- شکل 10: ساختار تمایز دهنده.....15
- شکل 11: ساختار دسته بند.....16
- شکل 12: توضیحات ابرپارامتر ها در مقاله.....17
- شکل 13: روند آموزش مدل.....18
- شکل 14: نمودار های روند آموزش مدل.....18
- شکل 15: دقت بخش دسته بند.....19
- شکل 16: خروجی های تولید شده و برچسب پیشبینی شده برای آنها!.....20
- شکل 17: نمونه تصاویر دیتاست قبل و بعد ای پیش پردازش.....22
- شکل 18: ساختار مدل در مقاله.....24
- شکل 19: ساختار مدل پیاده شده.....25
- شکل 20: رابطه Binary Cross Entropy.....26
- شکل 21: رابطه MSE.....26
- شکل 22: رابطه KL Divergence.....27
- شکل 23: رابطه تابع هزینه نهایی در مقاله.....27
- شکل 24: نمودار تابع هزینه مدل در حین ۵۰۰۰ اپیاک آموزش!.....28
- شکل 25: نمودار تابع هزینه مدل دوم در حین ۵۰۰ اپیاک آموزش.....29
- شکل 26: تصاویر تولید شده توسط مدل آموزش دیده با ۵۰۰ اپیاک.....30
- شکل 27: تصاویر تولید شده توسط مدل آموزش دیده با ۵۰۰۰ اپیاک.....30

- شکل 28: تصاویر بازسازی شده توسط مدلی که ۵۰۰۰ اپیاک آموزش دیده.... 31
- شکل 29: تصاویر بازسازی شده مدلی که ۵۰۰ اپیاک آموزش دیده..... 31
- شکل 30: رابطه PSNR..... 32
- شکل 31: توضیح SSIM..... 33
- شکل 32: حاصل معیارهای PSNR و SSIM برای مدلی که در ۵۰۰۰ اپیاک آموزش دیده..... 33
- شکل 33: حاصل معیارهای PSNR و SSIM برای مدلی که در ۵۰۰ اپیاک آموزش دیده..... 33
- شکل 34: رابطه PSNR..... 35
- شکل 35: توضیح SSIM..... 36
- شکل 36: روند آموزش CNN..... 38
- شکل 37: دقت مدل CNN..... 38

جدول ها

قبل از پاسخ دادن به پرسش‌ها، موارد زیر را با دقت مطالعه نمایید:

- از پاسخ‌های خود یک گزارش در قالبی که در صفحه‌ی درس در سامانه‌ی Elearn با نام [REPORTS_TEMPLATE.docx](#) قرار داده شده تهیه نمایید.
- پیشنهاد می‌شود تمرین‌ها را در قالب گروه‌های دو نفره انجام دهید. (بیش از دو نفر مجاز نیست و تحویل تک نفره نیز نمره‌ی اضافی ندارد) توجه نمایید الزامی در یکسان ماندن اعضای گروه تا انتهای ترم وجود ندارد. (یعنی، می‌توانید تمرین اول را با شخص A و تمرین دوم را با شخص B و ... انجام دهید)
- **کیفیت گزارش شما در فرآیند تصحیح از اهمیت ویژه‌ای برخوردار است؛** بنابراین، لطفاً تمامی نکات و فرضیه‌ی را که در پیاده‌سازیها و محاسبات خود در نظر می‌گیرید در گزارش ذکر کنید.
- در گزارش خود مطابق با آنچه در قالب نمونه قرار داده شده، برای شکل‌ها زیرنویس و برای جدول‌ها بالانویس در نظر بگیرید.
- الزامی به ارائه توضیح جزئیات کد در گزارش نیست، اما باید نتایج بدست آمده از آن را گزارش و تحلیل کنید.
- **تحلیل نتایج الزامی می‌باشد، حتی اگر در صورت پرسش اشاره‌ای به آن نشده باشد.**
- **دستیاران آموزشی ملزم به اجرا کردن کدهای شما نیستند؛** بنابراین، هرگونه نتیجه و یا تحلیلی که در صورت پرسش از شما خواسته شده را به طور واضح و کامل در گزارش بیاورید. در صورت عدم رعایت این مورد، بدیهی است که از نمره تمرین کسر میشود.
- **کدها حتماً باید در قالب نوت‌بوک با پسوند .ipynb تهیه شوند، در پایان کار، تمامی کد اجرا شود و خروجی هر سلول حتماً در این فایل ارسالی شما ذخیره شده باشد.** بنابراین برای مثال اگر خروجی سلولی یک نمودار است که در گزارش آورده‌اید، این نمودار باید هم در گزارش هم در نوت‌بوک کدها وجود داشته باشد.
- **در صورت مشاهده‌ی تقلب امتیاز تمامی افراد شرکت‌کننده در آن، 100- لحاظ میشود.**
- تنها زبان برنامه نویسی مجاز **Python** است.
- **استفاده از کدهای آماده برای تمرینها به هیچ وجه مجاز نیست.**
- **در صورتی که دو گروه از یک منبع مشترک استفاده کنند و کدهای مشابه تحویل دهند، تقلب محسوب می‌شود.**

- نحوه محاسبه تاخیر به این شکل است: پس از پایان رسیدن مهلت ارسال گزارش، حداکثر تا یک هفته امکان ارسال با تاخیر وجود دارد، پس از این یک هفته نمره آن تکلیف برای شما صفر خواهد شد.

0 سه روز اول: بدون جریمه

0 روز چهارم: ۵ درصد

0 روز پنجم: ۱۰ درصد

0 روز ششم: ۱۵ درصد

0 روز هفتم: ۲۰ درصد

- حداکثر نمره‌ای که برای هر سوال می‌توان اخذ کرد ۱۰۰ بوده و اگر مجموع بارم یک **سوال** بیشتر از ۱۰۰ باشد، در صورت اخذ نمره بیشتر از ۱۰۰، اعمال نخواهد شد.

0 برای مثال: اگر نمره اخذ شده از سوال ۱ برابر ۱۰۵ و نمره سوال

۲ برابر ۹۵ باشد، نمره نهایی تمرین ۹۷.۵ خواهد بود و نه ۱۰۰.

- لطفا گزارش، کدها و سایر ضمایم را به در یک پوشه با نام زیر قرار داده و آن را فشرده سازید، سپس در سامانه‌ی Elearn بارگذاری نمایید:

HW[Number]_[Lastname]_[StudentNumber]_[Lastname]_[StudentNumber].zip

(مثال: HW1_Ahmadi_810199101_Bagheri_810199102.zip)

- برای گروه‌های دو نفره، بارگذاری تمرین از جانب یکی از اعضا کافی است ولی پیشنهاد می‌شود هر دو نفر بارگذاری نمایند.

پرسش 1. یادگیری بدون نظارت و انتقال دامنه با استفاده از GAN

۱-۱. مقدمه

در این سوال قصد داریم با استفاده از GAN، انتقال دامنه انجام دهیم که برگرفته از کاری است که در این مقاله انجام شده است (<https://arxiv.org/abs/1612.05424>).

به طور خلاصه، از GAN استفاده می‌کنیم تا تصاویر یک دیتاست را به دیتاست دیگری تطبیق دهیم. در این تمرین، از دیتاست MNIST که سیاه و سفید است استفاده می‌کنیم و آن را به MNIST-M که شامل ارقام رنگی با پس‌زمینه‌های متنوع است، انتقال می‌دهیم.

۱-۲. بخش نظری

ابتدا به چند سؤال پاسخ خواهیم داد.

۱-۲-۱. ناپایداری در GAN

۱-۲-۱-۱. چرا آموزش GAN ناپایدار است؟

آموزش GAN ناپایدار است زیرا برخلاف سایر شبکه‌های عصبی، یک مسئله بهینه‌سازی ساده نیست، بلکه باید یک تعادل بین دو شبکه عصبی که در مقابل هم قرار دارند برقرار شود. در این فرایند، یک شبکه مولد (Generator) را در برابر یک شبکه تمایزدهنده (Discriminator) قرار می‌دهیم و این دو باید با یکدیگر رقابت کنند و به مرور بهتر شوند. اما اگر یکی از آن‌ها بیش از حد قوی‌تر از دیگری شود، کل فرایند شکست خواهد خورد.

۱-۲-۱-۲. دلایل اصلی ناپایداری

سه عامل اصلی ناپایداری در آموزش GAN عبارت‌اند از:

۱. فروپاشی مُد (Mode Collapse)

این یکی از رایج‌ترین خطاها در GAN است و زمانی رخ می‌دهد که مولد (Generator) فقط یک یا چند خروجی خاص را پیدا می‌کند که به خوبی می‌توانند تمایزدهنده (Discriminator) را فریب دهند. سپس فقط نمونه‌هایی از همان کلاس‌های خاص را تولید می‌کند و سایر داده‌ها را نادیده می‌گیرد. برای مثال، اگر بخواهیم GAN ما تصاویر ارقام تولید کند، ممکن است متوجه شود که تولید عدد «۱» آسان‌تر است و فقط شروع به تولید «۱» کند و در نهایت فقط عدد «۱» را به خوبی تولید کند و بقیه ارقام را نادیده بگیرد.

این مشکل به این دلیل به وجود می‌آید که هدف مولد فریب دادن تمایزدهنده است و اگر نقطه ضعیفی در آن پیدا کند، انگیزه‌ای برای کشف روش‌های دیگر یا تولید کلاس‌های دیگر نخواهد داشت.

۲. گرادیان محو شونده (Vanishing Gradient)

این حالت زمانی رخ می‌دهد که تمایزدهنده خیلی سریع یاد می‌گیرد که داده‌های واقعی و تقلبی را از هم تشخیص دهد. در این صورت، دقت آن به نزدیکی ۱۰۰٪ می‌رسد، اما گرادیان خطای آن تقریباً صفر می‌شود. برای اینکه مولد یاد بگیرد چگونه تمایزدهنده را فریب دهد، باید گرادیان‌های تمایزدهنده به آن منتقل شوند. اما اگر این گرادیان‌ها تقریباً صفر شوند (محو شوند!)، مولد اطلاعات کافی برای یادگیری دریافت نمی‌کند و نمی‌تواند پیشرفت کند. در نتیجه، فرآیند یادگیری ناپایدار می‌شود.

۳. همگرا نشدن (Non-convergence)

این حالت زمانی رخ می‌دهد که به جای رسیدن به یک نقطه پایدار که در آن مولد و تمایزدهنده هر دو عملکرد خوبی دارند، مدل وارد یک چرخه می‌شود که در آن مقادیر خطا (Loss) دائماً نوسان می‌کنند و هیچ‌گاه به تعادل نمی‌رسند.

۱-۲-۱-۳. راهکارهای مقابله با ناپایداری

۱. تصمیمات معماری

انتخاب معماری مناسب می‌تواند نقش بسیار مهمی در پایداری مدل داشته باشد. برای مثال، استفاده از شبکه‌های GAN مبتنی بر کانولوشن (مانند مدلی که در مقاله استفاده شده) ممکن است فرآیند آموزش را پایدارتر کند.

۲. تابع هزینه (Loss Function)

استفاده از تابع هزینه مینیمکس اصلی (Minimax Loss) - که با GAN معرفی شد - ممکن است در برخی کاربردها باعث ناپایداری شود. بنابراین استفاده از گزینه‌های جایگزین مانند "Wasserstein Loss" (که با استفاده از یک معیار متفاوت برای اندازه‌گیری فاصله بین توزیع‌ها از محو شدن گرادیان‌ها جلوگیری می‌کند) یا "Least-Squares Loss" (که نمونه‌های دور از مرز تصمیم‌گیری را بیشتر جریمه می‌کند و اطلاعات بهتری به مولد در مورد چگونگی بهبود می‌دهد) می‌تواند کمک‌کننده باشد.

۳. Regularization و راهبردهای آموزشی

استفاده از توابع هزینه خاص برای وظایف خاص (مانند آنچه در مقاله آمده)، تنظیم‌سازی (Regularization) یا روش‌های دیگر مانند افزودن نویز به

ورودی یا دادن برچسب‌ها به تمایزدهنده نیز می‌توانند هنگام مواجهه با ناپایداری در آموزش مؤثر باشند.

۱-۲-۲. چرا مقاله فقط از نویز به عنوان ورودی مولد استفاده نمی‌کند؟

در یک GAN معمولی، مولد فقط از یک بردار نویز تصادفی به عنوان ورودی استفاده می‌کند و یاد می‌گیرد که چگونه آن را به خروجی‌های قابل قبول تبدیل کند. این روش برای تولید نمونه‌های جدید از یک دیتاست مناسب است، اما اگر بخواهیم یک تصویر را از یک حوزه (Domain) به حوزه‌ای دیگر منتقل کنیم، به رویکرد متفاوتی نیاز داریم.

صرف استفاده از بردار نویز، کنترل کمی روی خروجی می‌دهد. به همین دلیل مقاله، GAN را روی یک تصویر از حوزه‌ای که می‌خواهیم آن را تبدیل کنیم شرطی می‌کند. این کار به مدل اجازه می‌دهد تا محتوا را از سبک (Style) جدا کند. در طول مرحله یادگیری، مدل می‌آموزد که تصویر ورودی، محتوا را فراهم می‌کند (در اینجا یعنی مشخص می‌کند که عدد چیست) و نویز، تغییرات سبک را کنترل می‌کند. به این ترتیب، می‌توانیم تعداد نامحدودی نمونه با سبک‌های متفاوت ولی محتوای مشابه با تصویر ورودی تولید کنیم.

۱-۲-۳. اجزای مدل

۱-۲-۳-۱. نقش اجزای مدل

مدلی که در مقاله معرفی شده، سه جزء اصلی دارد: (مطابق شکل زیر)

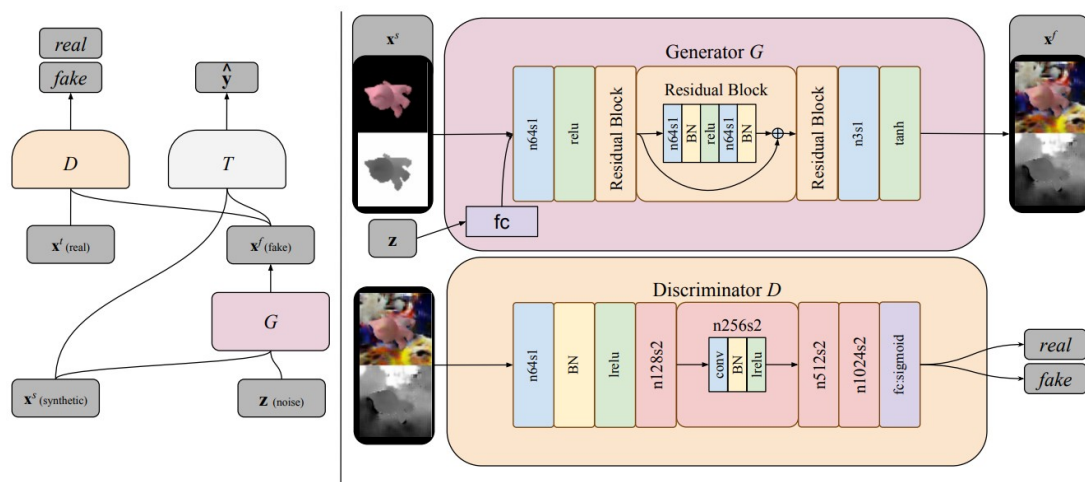


Figure 2. An overview of the model architecture. On the left, we depict the overall model architecture following the style in [34]. On the right, we expand the details of the generator and the discriminator components. The generator G generates an image conditioned on a synthetic image $\mathbf{x}^s$ and a noise vector $\mathbf{z}$. The discriminator D discriminates between real and fake images. The task-specific classifier T assigns task-specific labels $\mathbf{y}$ to an image. A convolution with stride 1 and 64 channels is indicated as $n64s1$ in the image. lrelu stands for leaky ReLU nonlinearity. BN stands for a batch normalization layer and FC for a fully connected layer. Note that we are not displaying the specifics of T as those are different for each task and decoupled from the domain adaptation process.

شکل 1: ساختار مدل مقاله

۱. مولد (Generator)

این بخش نقش تبدیل کننده تصویر را دارد. وظیفه آن این است که یک تصویر از حوزه مبدأ (در اینجا MNIST) به همراه یک بردار نویز دریافت کند و آن را به تصویری تبدیل کند که طوری به نظر برسد که انگار از حوزه هدف (در اینجا MNIST-M) برداشته شده و بتواند تمایزدهنده را فریب دهد.

۲. تمایزدهنده (Discriminator)

وظیفه تمایزدهنده بررسی واقعی یا جعلی بودن تصاویر است. وقتی تصویری به آن داده می شود، آن را به صورت "واقعی" یا "جعلی" برچسب گذاری می کند و سعی می کند تصاویر جعلی تولیدشده توسط مولد را تشخیص دهد. بازخوردی که از تمایزدهنده دریافت می شود به مولد کمک می کند تا تصاویر بهتری تولید کند.

۳. دسته بند (Classifier)

این بخش مدلی است که وظیفه انجام کار موردنظر ما را بر عهده دارد (در اینجا تشخیص رقم ها در تصاویر). این مدل هم روی تصاویر واقعی از حوزه هدف و هم روی تصاویر جعلی تولیدشده توسط مولد آموزش می بیند. این کار باعث می شود مولد تصاویری تولید کند که نه تنها ظاهر خوبی دارند، بلکه محتوا را نیز حفظ می کنند (مثلاً اطمینان حاصل می شود که در طی فرایند تبدیل، عدد 1 به 7 تبدیل نشود!).

۱-۲-۳-۲. عملکرد مدل در صورت حذف هر یک از اجزا چگونه تغییر می‌کند؟

اگر مولد (Generator) حذف شود، کل فرآیند غیرممکن خواهد بود، زیرا این بخش عنصر اصلی در تبدیل بین حوزه‌هاست. بدون آن، فقط می‌توانیم یک دسته‌بند روی حوزه مبدأ آموزش دهیم که به دلیل تفاوت بین حوزه‌ها (Domain Shift) عملکرد ضعیفی خواهد داشت (همان‌طور که در مقاله گفته شده، این مدل زمانی استفاده می‌شود که داده‌های برچسب‌دار از حوزه هدف در دسترس نباشند یا تهیه آن‌ها پرهزینه باشد).

اگر تمایزدهنده (Discriminator) حذف شود، مولد بازخورد خود را از دست می‌دهد و بنابراین هیچ انگیزه‌ای برای تولید تصاویری که به نظر برسد متعلق به حوزه هدف هستند نخواهد داشت. در این صورت، مولد فقط توسط دسته‌بند آموزش می‌بیند و یاد می‌گیرد تصاویری تولید کند که دسته‌بند بتواند آن‌ها را تشخیص دهد، ولی این تصاویر به احتمال زیاد شبیه سبک دیتاست هدف (MNIST-M) نخواهند بود.

اگر دسته‌بند (Classifier) حذف شود، مولد فقط برای فریب دادن تمایزدهنده آموزش می‌بیند. در این حالت، مولد همچنان یاد می‌گیرد تصاویری تولید کند که به نظر برسند از حوزه هدف (MNIST-M) هستند، اما هیچ انگیزه‌ای برای حفظ محتوای تصویر ورودی نخواهد داشت. از آنجا که خروجی‌ها صرفاً از نظر ظاهری به حوزه هدف شباهت دارند، ممکن است مثلاً عدد «1» را به «7» با سبک MNIST-M تبدیل کند. در حالی که هدف این مدل تولید نمونه‌هایی از حوزه هدف است که برچسب (محتوا) آن‌ها حفظ شده باشد تا در وظایف دیگر قابل استفاده باشند.

۱-۲-۴. تابع هزینه Content similarity

در این مقاله یک تابع هزینه به نام Content similarity معرفی شده است که نوع خاصی از تابع خطا به نام Masked Pairwise Mean Squared Error - PMSE است و هدف آن اطمینان از این است که محتوای تصویر تولیدشده (در اینجا رقم) با تصویر ورودی یکسان باشد.

۱-۲-۴-۱. چرا این تابع هزینه ضروری است؟

هدف مقاله طراحی یک تبدیل بین دو حوزه برای آموزش یک مدل (دسته‌بند Classifier) است و این موضوع نیاز دارد که مدل مولد برچسب‌ها را حفظ کند. اگر مولد محتوای تصویر ورودی را تغییر دهد، برچسب‌ها دیگر معتبر نخواهند بود و اگر این داده‌های اشتباه برچسب‌خورده را به دسته‌بند بدهیم، عملکرد آن به شدت کاهش خواهد یافت.

۱-۲-۴-۲. رفتار مولد در صورت حذف این تابع هزینه

اگر تابع هزینه‌ی Content Similarity حذف شود، مولد فقط توسط تابع هزینه Adversarial و تابع هزینه مربوط به وظیفه (Task Loss) از طرف دسته‌بند آموزش می‌بیند. این وضعیت ممکن است باعث شود که مولد برای تطبیق بهتر با حوزه هدف، تغییرات بیشتری در محتوای تصویر ورودی اعمال کند، که در نتیجه منجر به تغییر در کلاس شود، همان‌طور که در مقاله نیز ذکر شده است.

علاوه بر این، حذف این تابع هزینه می‌تواند باعث ناپایداری در آموزش شود، چرا که این تابع می‌تواند به عنوان نوعی تنظیم‌گر (Regularizer) عمل می‌کند و کمک کند مولد بین حفظ محتوا و تغییر سبک تعادل برقرار کند.

۱-۲-۵. چرا فقط از تصاویر تولیدشده برای آموزش دسته‌بند استفاده نمی‌کنیم؟

دلیل اصلی این رویکرد، وادار کردن مولد به حفظ محتوای تصاویر است. اگر ما دسته‌بند را فقط با تصاویر تولیدشده آموزش دهیم، ممکن است کلاس‌ها را به اشتباه یاد بگیرد. برای مثال، اگر مولد عدد 1 را طوری تولید کند که شبیه عدد 7 باشد و دسته‌بند فقط تصاویر جعلی را ببیند، ممکن است تصور کند که کلاس برچسب‌خورده به عنوان 1، شامل رقم‌هایی شبیه به 7 است!

در واقع این روش از تغییر محتوای تصاویر توسط مولد جلوگیری می‌کند و آن را مجبور می‌سازد فقط سبک (Style) را تغییر دهد نه محتوا (Content).

علاوه بر این، افزودن محدودیت‌هایی از این نوع به مسئله بهینه‌سازی می‌تواند موجب پایداری بیشتر آموزش شود، زیرا مانند یک تنظیم‌گر (Regularizer) عمل می‌کند.

۱-۲-۶. آیا این روش برای حوزه‌هایی با تفاوت معنایی (Semantic) قابل استفاده است؟

خیر، این روش به احتمال زیاد در چنین حوزه‌هایی شکست می‌خورد؛ چرا که کل رویکرد آن بر این فرض استوار است که محتوای اصلی بین دو حوزه یکسان است و تنها تفاوت آن‌ها در سبک (Style) است. به بیان دیگر، این روش اساساً نوعی تکنیک انتقال سبک (Style Transfer) محسوب می‌شود.

تابع هزینه Content Similarity و آموزش دسته‌بند روی تصاویر واقعی و تولیدشده (هر دو)، همگی با هدف وادار کردن مولد به حفظ برچسب‌ها انجام

می‌شوند. بنابراین اگر مولد قرار باشد برچسب‌ها را تغییر دهد، هیچ‌کدام از این روش‌ها دیگر کاربردی نخواهند داشت.

برای مثال، در حالتی که اشیاء متفاوت باشند (مثلاً "سگ" در برابر "گربه")، هیچ راه منطقی برای تبدیل تصویر سگ به گربه وجود ندارد. یا در شرایطی که تفاوت‌ها در زاویه دید باشد، مولد نیاز به درک ساختار سه‌بعدی تصویر خواهد داشت، که این موضوع به تغییرات اساسی در معماری شبکه نیاز دارد.

همچنین در زمینه ترجمه زبان، معمولاً از GAN استفاده نمی‌شود؛ بلکه روش‌های رایج‌تری که شامل شبکه‌های بازگشتی (RNN) یا مدل‌های ترنسفورمر هستند به کار می‌روند.

۱-۳. بخش عملی

اکنون مدل معرفی‌شده در مقاله را پیاده‌سازی خواهیم کرد.

۱-۳-۱. پیش‌پردازش داده‌ها

در این بخش، دیتاست‌های MNIST و MNIST-M را بارگذاری کرده و مراحل لازم برای پیش‌پردازش را انجام می‌دهیم.

۱-۳-۱-۱. بارگذاری و نمایش

در ابتدا دیتاست‌ها را بارگذاری کرده و ۵ جفت تصویر متناظر از آن‌ها را نمایش می‌دهیم.

* از آنجا که فایل‌های pkl مربوط به نسخه‌ای قدیمی از پایتون بودند، مجبور شدم از یک تابع (که پیدایش کردم) برای خواندن آن‌ها استفاده کنم!



شکل 2: نمونه تصاویر دیتاست‌های MNIST و MNIST-M

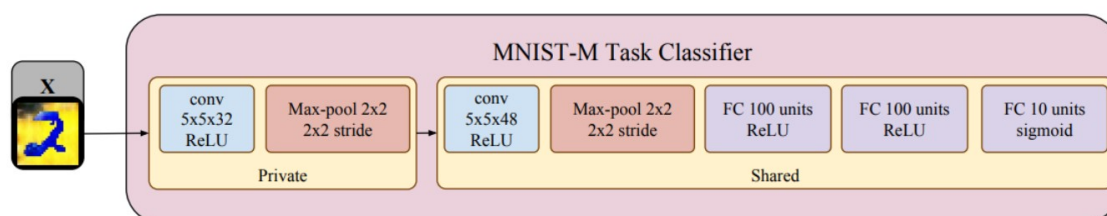
۱-۳-۱-۲. آماده‌سازی و تقسیم داده‌ها برای شبکه

در این مرحله، داده‌ها را به دیتالودرهای جداگانه تقسیم می‌کنیم. برای هماهنگی با دیتاست MNIST-M، ابتدا تصاویر خاکستری MNIST را به تصاویر RGB تبدیل می‌کنیم، سپس آن‌ها را مطابق مقاله به اندازه 32×32 تغییر اندازه داده و در نهایت مقدار پیکسل‌ها را به بازه $[-1, 1]$ نرمال‌سازی می‌کنیم.

۱-۳-۲. مشاهده فاصله بین دو حوزه (Domain Gap)

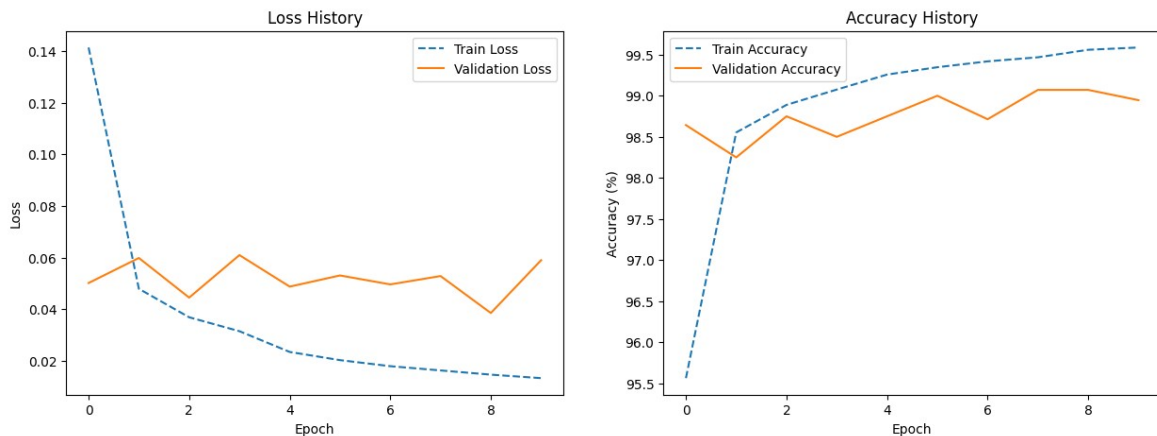
در این بخش، یک دسته‌بند طبق ساختاری که در مقاله آمده پیاده‌سازی خواهیم کرد (که در تصویر زیر قابل مشاهده است) و آن را بر روی دیتاست MNIST آموزش می‌دهیم. سپس آن را روی بخش تست دیتاست MNIST و همچنین کل دیتاست MNIST-M آزمایش می‌کنیم تا فاصله بین دو حوزه را ببینیم. این اختلاف عملکرد به عنوان معیاری برای ارزیابی مدل GAN ما خواهد بود (که انتظار داریم دسته بندی آموزش داده شده روی خروجی های GAN عملکرد بهتری نسبت به این داشته باشد).

هایپیرامترها مطابق با مقادیر انتخاب شده در مقاله در نظر گرفته شده‌اند.



شکل 3: ساختار دسته بند (*Classifire*) در مقاله

در ابتدا توابع لازم را تعریف خواهیم کرد و سپس مدل را آموزش داده و روند دقت (اعتبارسنجی) و خطای آن را در طول آموزش ترسیم می‌کنیم.



شکل 4: نمودارهای خطا و دقت مدل روی داده های مبدا (MNIST) در حین آموزش

همان طور که مشاهده می کنیم، مدل به خوبی آموزش دیده و توانسته به دقت ۹۹٪ روی مجموعه داده اعتبارسنجی برسد! در مرحله بعد، آن را روی داده های تست MNIST و همچنین کل دیتاست MNIST-M آزمایش می کنیم.

Accuracy on MNIST test set: 98.84%
Accuracy on full MNIST-M dataset: 60.99%

شکل 5: دقت مدل روی داده های تست MNIST و کل MNIST-M

نتایج نشان می دهند که عملکرد مدل روی مجموعه تست MNIST برابر با ۹۸٪ بوده که نشان دهنده آموزش مؤثر مدل است، اما دقت آن روی دیتاست MNIST-M تنها ۶۰٪ بوده است که این مسئله وجود فاصله (Domain Shift) بین این دو دیتاست را نشان می دهد.

۱-۳-۳. پیاده سازی معماری مدل

در این بخش، سه مؤلفه اصلی مدل را پیاده سازی خواهیم کرد: مولد (Generator یا G)، تمایز دهنده (Discriminator یا D) و دسته بند (Task Classifier یا T)، بر اساس توضیحات و تصاویر ارائه شده در مقاله که در شکل های زیر قابل مشاهده هستند.

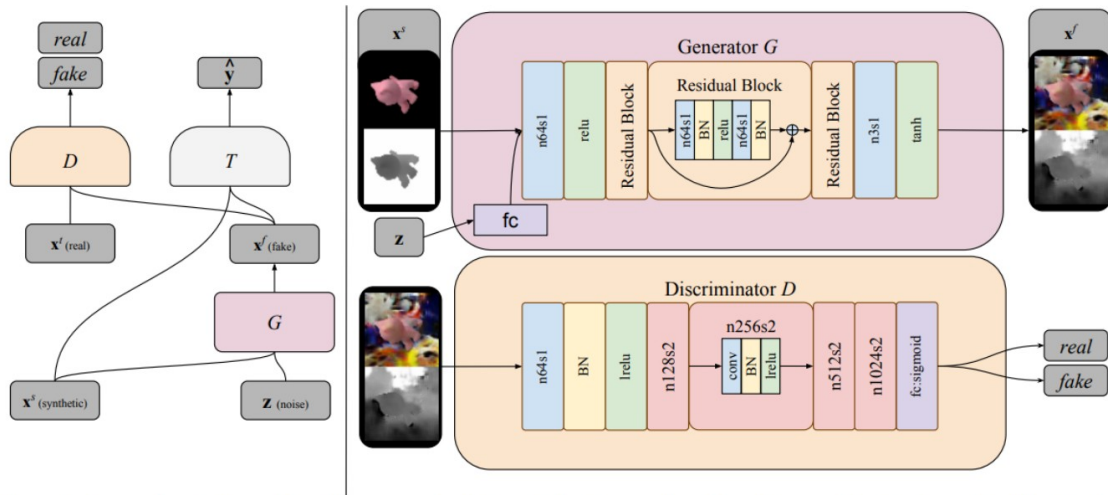


Figure 2. An overview of the model architecture. On the left, we depict the overall model architecture following the style in [34]. On the right, we expand the details of the generator and the discriminator components. The generator G generates an image conditioned on a synthetic image x^s and a noise vector z . The discriminator D discriminates between real and fake images. The task-specific classifier T assigns task-specific labels y to an image. A convolution with stride 1 and 64 channels is indicated as $n64s1$ in the image. lrelu stands for leaky ReLU nonlinearity. BN stands for a batch normalization layer and FC for a fully connected layer. Note that we are not displaying the specifics of T as those are different for each task and decoupled from the domain adaptation process.

شکل 6: نمودار ساختار کلی مدل

B. Model Architectures and Parameters

We present the exact model architectures used for each experiment along with hyperparameters needed to reproduce results. The general form for the generator, G , and discriminator, D , are depicted in Figure 2 of the paper. For G , we vary the number of filters and the number of residual blocks. For D , we vary the amount of regularization and the number of layers.

Optimization consists of alternating optimization of the discriminator and task classifier parameters, referred to as the D step, with optimization of the generator parameters, referred to as the G step.

Unless otherwise specified, the following hyperparameters apply to all experiments:

- Batch size 32
- Learning rate decayed by 0.95 every 20,000 steps
- All convolutions have a 3×3 filter kernel
- Inject noise drawn from a zero centered Gaussian with stddev 0.2 after every layer of discriminator
- Dropout every layer in discriminator with keep probability of 90%
- Input noise vector is 10 dimensional sampled from a uniform distribution $\mathcal{U}(-1, 1)$
- We follow conventions from the DCGAN paper [36] for several aspects
 - An L2 weight decay of $1e^{-5}$ is applied to all parameters
 - Leaky ReLUs have a leakiness parameter of 0.2
 - Parameters initialized from zero centered Gaussian with stddev 0.02
 - We use the ADAM optimizer with $\beta_1 = 0.5$

شکل 7: پارامترهای معماری مدل

B.2 MNIST-M Experiments (Paper Table 1)

Generator: The generator has 6 residual blocks with 64 filters each

Discriminator: The discriminator has 4 convolutions with 64, 128, 256, and 512 filters respectively. It has the same overall structure as paper Figure 2

Loss weights:

- Base learning rate is $1e^{-3}$
- The discriminator loss weight is 0.13
- The generator loss weight is 0.011
- The task classifier loss weight in G step is 0.01
- There is no similarity loss between the synthetic and generated images

شکل 8: جزییات مدل به کار رفته برای کار ما (MNIST-M)

۱-۳-۳-۱. مولد (Generator – G)

بر اساس مقاله، مولد دارای معماری شبیه به UNet است که شامل ۶ بلوک Residual Block و ۶۴ فیلتر در لایه‌های کانولوشنی خود می‌باشد. این مدل به صورت همزمان یک تصویر از دامنه مبدأ و یک بردار نویز ۱۰ بعدی را به عنوان ورودی دریافت می‌کند.

```

Generator(
  (fc_noise): Linear(in_features=10, out_features=1024, bias=True)
  (conv1): Sequential(
    (0): Conv2d(4, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): ReLU(inplace=True)
  )
  (res_blocks): Sequential(
    (0): ResidualBlock(
      (block): Sequential(
        (0): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): ReLU(inplace=True)
        (3): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (4): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      )
    )
    (1): ResidualBlock(
      (block): Sequential(
        (0): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): ReLU(inplace=True)
        (3): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (4): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      )
    )
    (2): ResidualBlock(
      (block): Sequential(
        (0): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): ReLU(inplace=True)
        (3): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (4): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      )
    )
    (3): ResidualBlock(
      (block): Sequential(
        (0): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): ReLU(inplace=True)
        (3): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (4): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      )
    )
    (4): ResidualBlock(
      (block): Sequential(
        (0): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): ReLU(inplace=True)
        (3): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (4): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      )
    )
    (5): ResidualBlock(
      (block): Sequential(
        (0): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (2): ReLU(inplace=True)
        (3): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (4): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      )
    )
  )
  (conv_out): Sequential(
    (0): Conv2d(64, 3, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): Tanh()
  )
)

```

شكل 9: ساختار مولد

۱-۳-۳-۲. تمایز دهنده (Discriminator – D)

تمایز دهنده یک شبکه کانولوشنی استاندارد است که وظیفه‌اش تشخیص واقعی یا جعلی بودن تصویر است (یعنی اینکه تصویر از دامنه هدف آمده یا توسط مولد تولید شده). این مدل دارای ۴ لایه کانولوشنی با اندازه فیلترهای ۶۴، ۱۲۸، ۲۵۶، ۵۱۲ است و در انتها یک لایه Fully Connected برای خروجی طبقه‌بندی دارد.

```
Discriminator(  
(model): Sequential(  
  (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
  (1): LeakyReLU(negative_slope=0.2, inplace=True)  
  (2): Dropout2d(p=0.1, inplace=False)  
  (3): Conv2d(64, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))  
  (4): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
  (5): LeakyReLU(negative_slope=0.2, inplace=True)  
  (6): Dropout2d(p=0.1, inplace=False)  
  (7): Conv2d(128, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))  
  (8): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
  (9): LeakyReLU(negative_slope=0.2, inplace=True)  
  (10): Dropout2d(p=0.1, inplace=False)  
  (11): Conv2d(256, 512, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))  
  (12): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
  (13): LeakyReLU(negative_slope=0.2, inplace=True)  
  (14): Dropout2d(p=0.1, inplace=False)  
)  
(adv_layer): Sequential(  
  (0): Linear(in_features=8192, out_features=1, bias=True)  
  (1): Sigmoid()  
)  
)
```

شکل 10: ساختار تمایز دهنده

۱-۳-۳-۳. دسته بند (Task Classifier – T)

این مدل همان معماری بخش قبلی را دارد اما برای سازگاری با ساختار GAN اندکی تغییر در آن داده شده است.

```

Task Classifier
TaskClassifier(
  (private_block): Sequential(
    (0): Conv2d(3, 32, kernel_size=(5, 5), stride=(1, 1))
    (1): ReLU()
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (shared_block_conv): Sequential(
    (0): Conv2d(32, 48, kernel_size=(5, 5), stride=(1, 1))
    (1): ReLU()
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (shared_block_fc): Sequential(
    (0): Linear(in_features=1200, out_features=100, bias=True)
    (1): ReLU()
    (2): Linear(in_features=100, out_features=100, bias=True)
    (3): ReLU()
    (4): Linear(in_features=100, out_features=10, bias=True)
  )
)

```

شکل 11: ساختار دسته بند

۴-۳-۱. توابع هزینه (Cost Functions)

فرآیند آموزش این مدل چند مرحله‌ای است و به صورت متناوب بین آموزش مولد (G) و آموزش متمایزکننده (D) و دسته‌بند (T) انجام می‌شود. در مرحله اول، مولد ثابت در نظر گرفته می‌شود و دو مؤلفه دیگر آموزش داده می‌شوند.

تمایز دهنده (D) تلاش می‌کند در تشخیص تصاویر واقعی از تصاویر جعلی تولیدشده توسط مولد بهتر شود. به همین منظور هم تصاویر واقعی از دامنه هدف و هم تصاویر تولیدشده توسط مولد را دریافت می‌کند و از تابع هزینه adversarial loss که همان **باینری کراس انتروپی (Binary Cross Entropy)** است، برای آموزش استفاده می‌کند.

دسته‌بند نیز تلاش می‌کند در تشخیص ارقام عملکرد بهتری داشته باشد و تصاویر واقعی از دامنه مبدأ (MNIST) و همچنین تصاویر تولیدی مولد را دریافت می‌کند تا مولد را مجبور کند محتوای تصویر (عدد) را حفظ کند. برای این کار از تابع هزینه task loss که **کراس انتروپی (Cross Entropy)** است، استفاده می‌شود.

در مرحله دوم، مولد به‌روزرسانی می‌شود تا در دو کار بهتر شود:

۱. فریب دادن تمایزدهنده (برای تطبیق با سبک دامنه هدف) با استفاده از

تابع هزینه Adversarial loss

۲. کمک به دسته‌بند برای عملکرد بهتر (حفظ محتوای تصویر) با استفاده از

تابع هزینه Task loss

این توابع هزینه با ضرایب از پیش تعیین‌شده (که یک ابرپارامتر است) با هم ترکیب می‌شوند که مقادیر آن‌ها در بخش B2 مقاله (تصویر زیر) قابل مشاهده است:

B.2 MNIST-M Experiments (Paper Table 1)

Generator: The generator has 6 residual blocks with 64 filters each

Discriminator: The discriminator has 4 convolutions with 64, 128, 256, and 512 filters respectively. It has the same overall structure as paper Figure 2

Loss weights:

- Base learning rate is $1e^{-3}$
- The discriminator loss weight is 0.13
- The generator loss weight is 0.011
- The task classifier loss weight in G step is 0.01
- There is no similarity loss between the synthetic and generated images

شکل 12: توضیحات ابرپارامترها در مقاله

در کد از تابع هزینه باینری کراس انتروپی برای Adversarial loss و از کراس انتروپی برای Task loss استفاده می‌کنیم.

۵-۳-۱. آموزش مدل

اکنون مدل را مطابق توضیح داده شده آموزش می‌دهیم (ابتدا آموزش تمایز دهنده، سپس آموزش دسته‌بند و در نهایت آموزش مولد). همچنین یک تابع به نام save_checkpoint تعریف می‌کنیم تا مدل را هر ۵ اپک ذخیره کنیم تا بتوانیم بهترین مدل را حفظ کنیم.

یک نکته مهم که باید به آن توجه شود این است که برای شبیه‌سازی شرایط دنیای واقعی، نباید در طول فرآیند آموزش، دقت (accuracy) و تابع هزینه (loss) را روی داده‌های تست ترسیم کنیم و نباید اجازه دهیم داده‌های تست بر روند آموزش مدل تأثیر بگذارند. اما چون هدف پروژه آموزشی است، نمودار آن‌ها را رسم کرده‌ام!


```

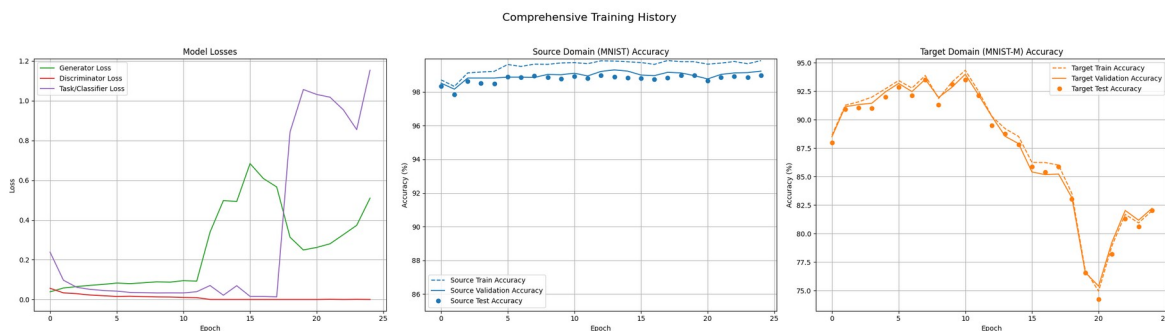
Starting Adversarial Training
[Epoch 1/25] [D loss: 0.0562] [G loss: 0.0384] [T loss: 0.2379] [Source Acc: 98.52%] [Target Acc: 88.50%]
[Epoch 2/25] [D loss: 0.0332] [G loss: 0.0576] [T loss: 0.0968] [Source Acc: 98.16%] [Target Acc: 91.14%]
[Epoch 3/25] [D loss: 0.0290] [G loss: 0.0648] [T loss: 0.0612] [Source Acc: 98.82%] [Target Acc: 91.32%]
[Epoch 4/25] [D loss: 0.0223] [G loss: 0.0711] [T loss: 0.0507] [Source Acc: 98.82%] [Target Acc: 91.45%]
[Epoch 5/25] [D loss: 0.0188] [G loss: 0.0758] [T loss: 0.0451] [Source Acc: 98.82%] [Target Acc: 92.43%]
Saved checkpoint for epoch 5 at checkpoints/checkpoint_epoch_5.pth
[Epoch 6/25] [D loss: 0.0152] [G loss: 0.0827] [T loss: 0.0418] [Source Acc: 98.88%] [Target Acc: 93.18%]
[Epoch 7/25] [D loss: 0.0163] [G loss: 0.0796] [T loss: 0.0354] [Source Acc: 98.88%] [Target Acc: 92.46%]
[Epoch 8/25] [D loss: 0.0143] [G loss: 0.0838] [T loss: 0.0344] [Source Acc: 98.86%] [Target Acc: 93.54%]
[Epoch 9/25] [D loss: 0.0128] [G loss: 0.0887] [T loss: 0.0327] [Source Acc: 99.04%] [Target Acc: 91.95%]
[Epoch 10/25] [D loss: 0.0121] [G loss: 0.0873] [T loss: 0.0331] [Source Acc: 99.02%] [Target Acc: 92.84%]
Saved checkpoint for epoch 10 at checkpoints/checkpoint_epoch_10.pth
[Epoch 11/25] [D loss: 0.0101] [G loss: 0.0945] [T loss: 0.0328] [Source Acc: 99.11%] [Target Acc: 93.98%]
[Epoch 12/25] [D loss: 0.0091] [G loss: 0.0922] [T loss: 0.0390] [Source Acc: 98.95%] [Target Acc: 92.16%]
[Epoch 13/25] [D loss: 0.0002] [G loss: 0.3388] [T loss: 0.0701] [Source Acc: 99.21%] [Target Acc: 90.27%]
[Epoch 14/25] [D loss: 0.0000] [G loss: 0.4979] [T loss: 0.0213] [Source Acc: 99.30%] [Target Acc: 88.54%]
[Epoch 15/25] [D loss: 0.0000] [G loss: 0.4930] [T loss: 0.0691] [Source Acc: 99.23%] [Target Acc: 87.89%]
Saved checkpoint for epoch 15 at checkpoints/checkpoint_epoch_15.pth
[Epoch 16/25] [D loss: 0.0000] [G loss: 0.6839] [T loss: 0.0155] [Source Acc: 99.00%] [Target Acc: 85.41%]
[Epoch 17/25] [D loss: 0.0000] [G loss: 0.6087] [T loss: 0.0156] [Source Acc: 98.96%] [Target Acc: 85.18%]
[Epoch 18/25] [D loss: 0.0000] [G loss: 0.5661] [T loss: 0.0130] [Source Acc: 99.16%] [Target Acc: 85.23%]
[Epoch 19/25] [D loss: 0.0000] [G loss: 0.3132] [T loss: 0.8433] [Source Acc: 99.12%] [Target Acc: 83.14%]
[Epoch 20/25] [D loss: 0.0000] [G loss: 0.2490] [T loss: 1.0565] [Source Acc: 98.96%] [Target Acc: 76.61%]
...
[Epoch 24/25] [D loss: 0.0007] [G loss: 0.3730] [T loss: 0.8548] [Source Acc: 99.14%] [Target Acc: 81.18%]
[Epoch 25/25] [D loss: 0.0001] [G loss: 0.5096] [T loss: 1.1534] [Source Acc: 99.23%] [Target Acc: 82.21%]
Saved checkpoint for epoch 25 at checkpoints/checkpoint_epoch_25.pth
Adversarial Training Finished

```

شکل 13: روند آموزش مدل

۱-۳-۶. نمایش نتایج

در این بخش ابتدا داده‌های ثبت‌شده در طول آموزش (دقت و تابع هزینه) را ترسیم می‌کنیم، سپس ۵ تصویر را همراه با خروجی مولد برای هر تصویر مربوطه و پیش‌بینی دسته‌بند برای همه آن‌ها نمایش می‌دهیم.



شکل 14: نمودارهای روند آموزش مدل

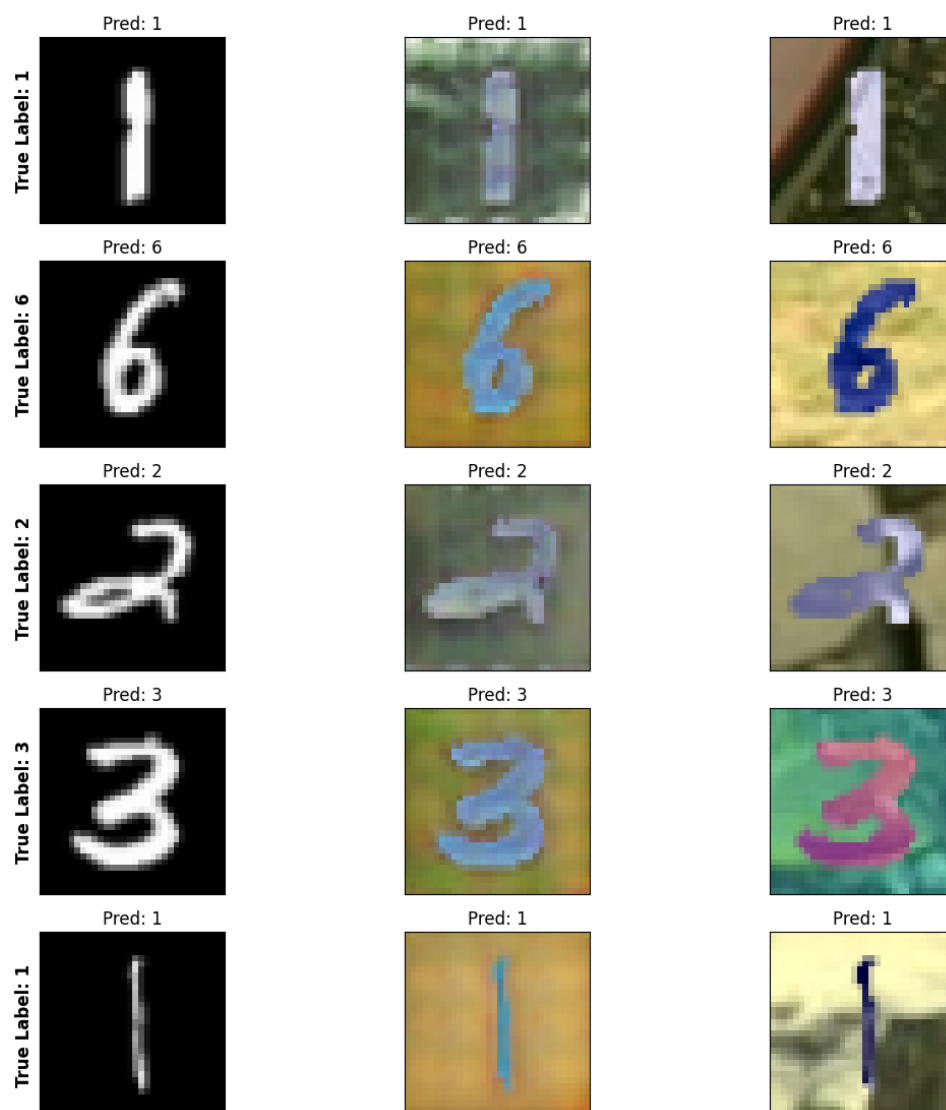
از نمودار میانی می‌توان دید که عملکرد دسته‌بند از همان ابتدا روی دامنه مبدأ (MNIST) بسیار خوب بوده (بیش از ۹۸٪). اما با نگاه به نمودار سمت راست مشخص می‌شود که عملکرد روی دامنه هدف (MNIST-M) چندان خوب نبوده و به تدریج بهبود یافته تا اینکه در ایپاک دهم، مدل دچار واگرایی شده و دقت روی دامنه هدف به شدت کاهش پیدا کرده است.

این مسئله را می‌توان با نگاه به نمودار تابع هزینه (نمودار سمت چپ) توضیح داد، جایی که می‌بینیم گرادیان تمایز دهنده تقریباً به صفر رسیده است (یعنی پدیده Gradient vanishing که پیش‌تر توضیح داده شد رخ داده است). بنابراین برای نمایش بهترین نتایج، ما مدل ذخیره‌شده بعد از اپاک دهم را بارگذاری کرده و خروجی‌های مولد را نمایش می‌دهیم.

```
Classifier accuracy:  
Accuracy on Source Train: 99.72%  
Accuracy on Source Validation: 99.02%  
Accuracy on Source Test: 98.79%  
Accuracy on Target Train: 93.30%  
Accuracy on Target Validation: 92.84%  
Accuracy on Target Test: 93.11%
```

شکل 15: دقت بخش دسته بند

Original vs. Generated vs. Real Target



شکل 16: خروجی های تولید شده و برجسب پیشبینی شده برای آنها!

همان طور که مشاهده می شود، تصاویر تولید شده توسط مولد بسیار خوب هستند، به خوبی به سبک مجموعه داده MNIST-M شباهت دارند و محتوای تصویر (عدد) نیز حفظ شده است.

دقت مدل نیز بسیار خوب است چه در داده آزمون دامنه مبدأ (با دقت ۹۸.۸٪) و چه در داده آزمون دامنه هدف (با دقت ۹۳٪).

پرسش ۲ – بازسازی تصاویر پولیپ آندوسکپی با EndoVAE

در این سوال قصد داریم کار انجام شده در این مقاله را بازتولید کنیم:

https://www.researchgate.net/publication/361927238_EndoVAE_Generating_Endoscopic_Images_with_a_Variational_Autoencoder

اما به جای استفاده از دیتاست مقاله، از دیتاست Kvasir موجود در سایت Kaggle استفاده خواهیم کرد:
(<https://www.kaggle.com/datasets/meetnagadia/kvasir-dataset>)

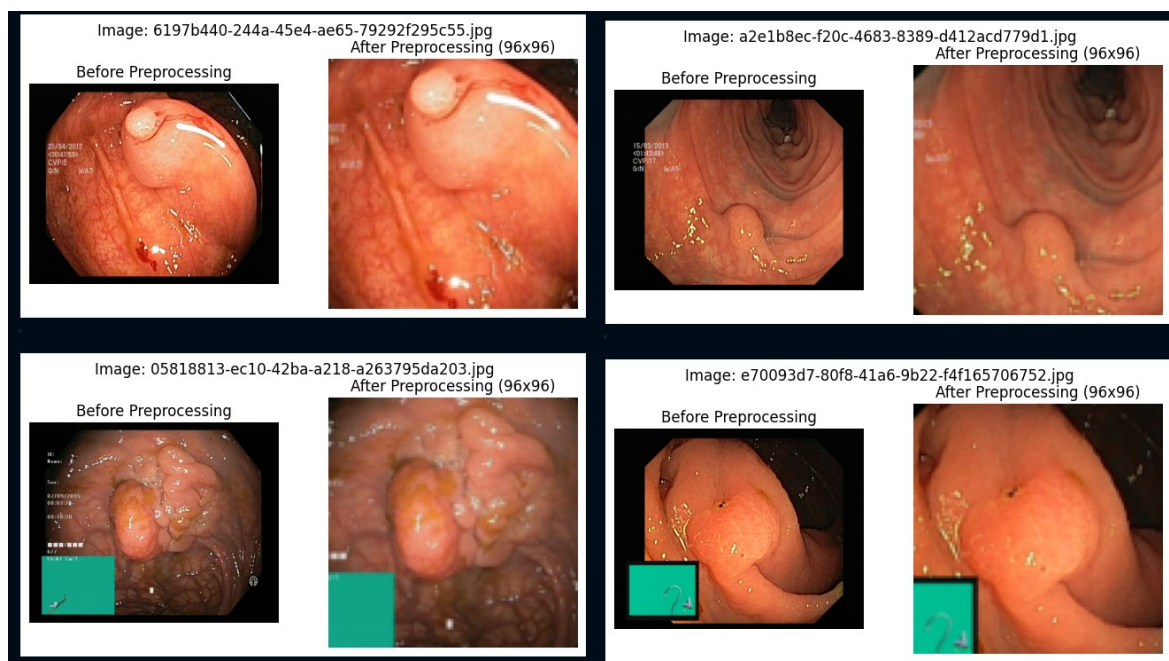
۲-۱. پیش‌پردازش داده‌ها

در گام اول، دیتاست را پیش‌پردازش می‌کنیم. بدین صورت که تصاویر سالم را از پوشه‌های normal-cecum و normal-z-line، normal-pylorus و تصاویر پلیپ را از پوشه polyp برداشته و فضای رنگ آن‌ها را به RGB تغییر می‌دهیم، ابعاد آن‌ها را با استفاده از درونیابی خطی (bilinear) به اندازه 96x96 تغییر می‌دهیم و سپس آن‌ها را به بازه [۰، ۱] نرمال‌سازی می‌کنیم.

همچنین با استفاده از روش‌های افزایش داده (Data Augmentation) تعداد عکس‌ها را بیشتر می‌کنیم.

در نهایت تصاویر پردازش شده را در پوشه‌ای به نام preprocessed و در دو زیردسته normal و polyp ذخیره خواهیم کرد.

همچنین چند نمونه از تصاویر را قبل و بعد از پیش‌پردازش برای بررسی نمایش می‌دهیم.



شکل 17: نمونه تصاویر دیتاست قبل و بعد از پیش پردازش.

۲-۲. معماری EndoVAE

در این بخش، معماری مدل EndoVAE که در مقاله معرفی شده است را توضیح داده و سپس آن را پیاده‌سازی می‌کنیم.

مانند سایر مدل‌های VAE، این مدل نیز از سه بخش اصلی تشکیل شده است:

۱. انکودر (Encoder)

انکودر یک تصویر با ابعاد بالا (در اینجا $96 \times 96 \times 3$) را به برداری کم‌بعد به نام فضای نهفته (latent space) تبدیل می‌کند. در این مدل، شش لایه کانولوشن دوبعدی (2D Convolution) نقش استخراج ویژگی را دارند. با عبور تصویر از این لایه‌ها، ابعاد آن کاهش می‌یابد (زمانی که stride بیشتر از ۱ باشد) و این موضوع باعث می‌شود مدل مجبور به یادگیری ویژگی‌های انتزاعی‌تر شود. پس از این مرحله، ویژگی‌ها وارد یک لایه خطی با ۲۵۶ نورون می‌شوند. در مرحله بعد، دو لایه خطی مجزا به نام‌های fc_mu و fc_logvar این ویژگی‌ها را به میانگین و لگاریتم واریانس یک توزیع نرمال نگاشت می‌کنند. این قسمت تفاوت کلیدی بین AutoEncoderهای معمولی و VAEهاست.

۲. بازپارامترسازی (Reparameterization)

این ترفند باعث می‌شود که VAE ها قابل آموزش با گرادیان شود. از آنجا که در VAE ما با توزیعی تصادفی به جای یک بردار ثابت سر و کار داریم، لازم است از این توزیع نمونه‌برداری کنیم. اما عملیات نمونه‌برداری تصادفی گرادیان ندارد.

ترفند بازپارامترسازی این مشکل را حل می‌کند: ابتدا یک بردار تصادفی از توزیع نرمال استاندارد (که هیچ پارامتر قابل یادگیری ای ندارد) نمونه‌برداری می‌کنیم، سپس این بردار را با انحراف معیار یادگرفته‌شده ضرب و با میانگین یادگرفته‌شده جمع می‌کنیم. به این ترتیب بخش تصادفی از بخش یادگرفتنی جدا شده و گرادیان قابل محاسبه می‌شود.

۳. دیکودر (Decoder)

دیکودر وظیفه ای معکوس انکودر دارد. یعنی با استفاده از بردار فضای نهفته تلاش می‌کند تصویر اولیه را بازسازی کند. ابتدا با یک لایه خطی، فضای نهفته را به یک تانسور بزرگ‌تر نگاشت کرده و یک نقشه ویژگی (feature map) تولید می‌کند. سپس با استفاده از لایه‌های transpose convolution (کانولوشن معکوس) به تدریج این نقشه ویژگی را آپ‌سَمپل کرده و ابعاد آن را افزایش می‌دهد. در پایان، یک لایه کانولوشن نهایی سه‌کاناله برای تولید تصویر RGB استفاده می‌شود.

در پیاده‌سازی، تمام لایه‌ها مطابق تصویر زیر و توضیحات بخش III مقاله تعریف شده‌اند.

نکته مهم: اگرچه در تصویر معماری ترکیبی از کانولوشن معمولی و معکوس به چشم می‌خورد، اما متن مقاله صراحتاً می‌گوید:

"The decoder part of the proposed model consists of a fully connected layer that contains 36,864 neurons followed by seven transpose convolutional layers with 256, 128, 64, 64, 32, 32 and 3 filters respectively"

بنابراین دیکودر را فقط با استفاده از لایه‌های transpose convolution پیاده‌سازی کرده‌ایم.

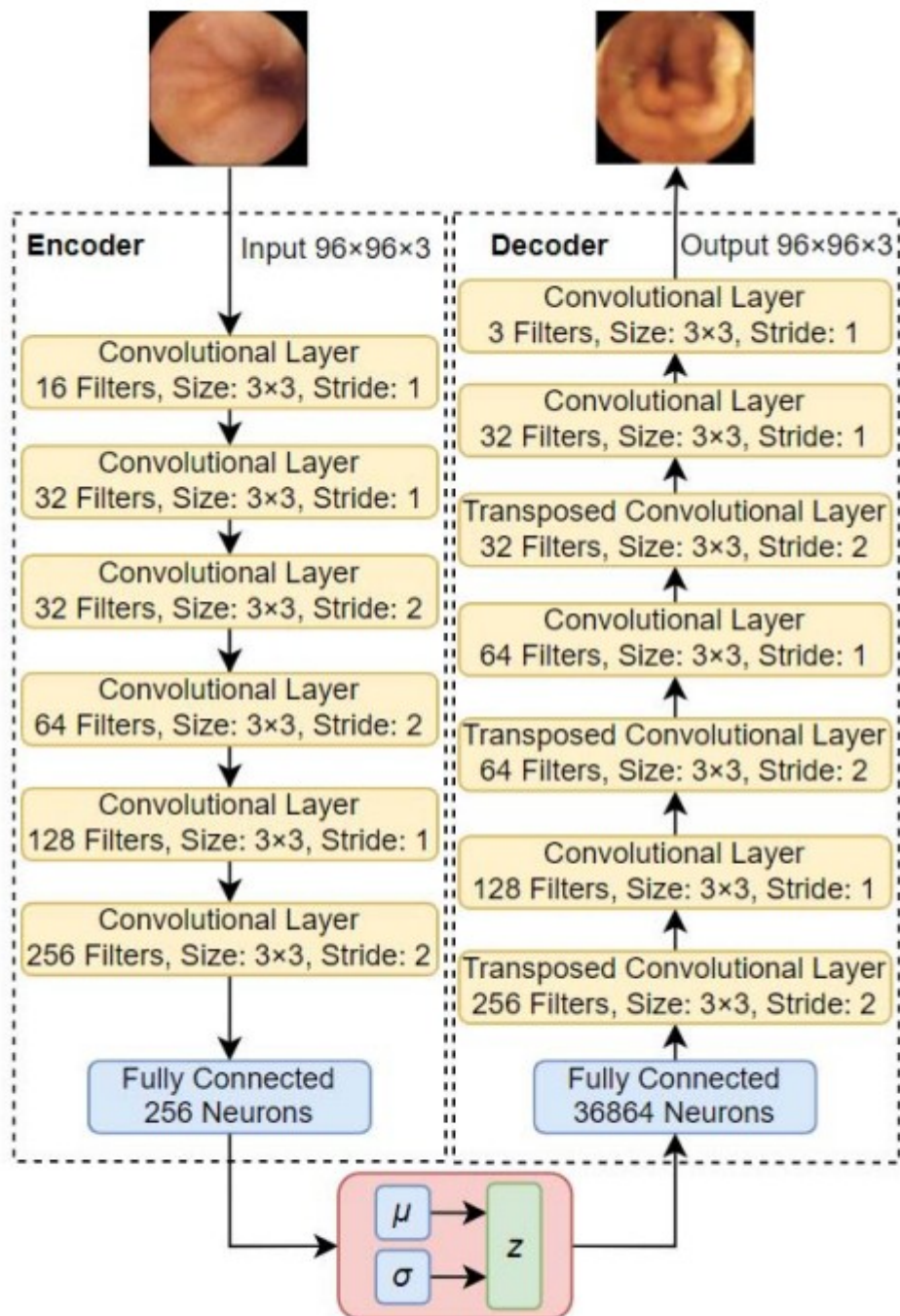


Fig. 1. Visual representation of the proposed EndoVAE methodology.

شکل 18: ساختار مدل در مقاله

```

EndoVAE(
  (encoder): Sequential(
    (0): Conv2d(3, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): ReLU()
    (2): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (3): ReLU()
    (4): Conv2d(32, 32, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))
    (5): ReLU()
    (6): Conv2d(32, 64, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))
    (7): ReLU()
    (8): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (9): ReLU()
    (10): Conv2d(128, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))
    (11): ReLU()
  )
  (fc_after_flatten): Linear(in_features=36864, out_features=256, bias=True)
  (fc_mu): Linear(in_features=256, out_features=6, bias=True)
  (fc_logvar): Linear(in_features=256, out_features=6, bias=True)
  (decoder_input): Linear(in_features=6, out_features=36864, bias=True)
  (decoder): Sequential(
    (0): ConvTranspose2d(256, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), output_padding=(1, 1))
    (1): ReLU()
    (2): ConvTranspose2d(256, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), output_padding=(1, 1))
    (3): ReLU()
    (4): ConvTranspose2d(128, 64, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), output_padding=(1, 1))
    (5): ReLU()
    (6): ConvTranspose2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (7): ReLU()
    (8): ConvTranspose2d(64, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (9): ReLU()
    (10): ConvTranspose2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (11): ReLU()
    (12): ConvTranspose2d(32, 3, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (13): Sigmoid()
  )
)

```

شکل 19: ساختار مدل پیاده شده

۲-۳. توابع هزینه

در این بخش قصد داریم توابع هزینه‌ای که در مقاله استفاده شده‌اند را توضیح داده و پیاده‌سازی کنیم.

برای EndoVAE از دو نوع زیان استفاده شده است:

۱. زیان بازسازی (Binary Cross-Entropy - BCE)

زیان بازسازی اندازه می‌گیرد که دیکودر تا چه حد می‌تواند تصویر اصلی را، پس از فشرده‌سازی در فضای نهفته، به درستی بازسازی کند. ما از زیان باینری کراس‌انتروپی (که فرمول آن در پایین آمده) استفاده می‌کنیم زیرا لایه نهایی در دیکودر ما یک سیگموید است. این تابع مقدار هر پیکسل خروجی را در بازه $[0, 1]$ خروجی می‌دهد که می‌تواند به عنوان احتمال تفسیر شود.

$$-\frac{1}{N} \sum_{i=1}^N y_i \cdot \log(p(y_i)) + (1 - y_i) \cdot \log(1 - p(y_i))$$

شکل 20: رابطه Binary Cross Entropy

- چرا از MSE استفاده نکردیم؟ در حالی که MSE (که با فرمول زیر محاسبه می‌شود) هم می‌تواند کار کند، در بازسازی تصویر زمانی که مقادیر پیکسل خروجی بین ۰ تا ۱ هستند، BCE معمولاً بهتر عمل می‌کند (در اکثر مواردی که بررسی کردم از BCE استفاده شده بود). این می‌تواند به این دلیل باشد که BCE مدل را زمانی که بسیار مطمئن است ولی اشتباه می‌کند، شدیدتر جریمه می‌کند.

$$MSE = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

شکل 21: رابطه MSE

۲. KL Divergence

این تابع هزینه نقطه تفاوتی است که VAE را به یک مدل مولد تبدیل می‌کند. Kullback-Leibler (KL) Divergence که با فرمول زیر محاسبه می‌شود، به عنوان یک منظم‌کننده (Regularizer) روی فضای نهفته عمل می‌کند. هدف آن این است که توزیع‌هایی که انکودر یاد می‌گیرد، تا حد امکان به یک توزیع نرمال استاندارد نزدیک باشند. بدون این قسمت انکودر ممکن است داده‌های آموزشی را به خاطر بسپارد و برای هر تصویر یک نقطه جداگانه در فضای نهفته قرار دهد در حالی که ما می‌خواهیم فضای نهفته شبیه یک توزیع گاوسی باشد تا بتوانیم بعدها از آن نمونه‌برداری کنیم و تصاویر جدیدی تولید کنیم که شبیه به مجموعه داده‌ی اصلی باشند.

$$D_{KL}(p(x) || q(x)) = \sum_{x \in X} p(x) \ln \frac{p(x)}{q(x)}$$

شکل 22: رابطه KL Divergence

این دو زیان سپس در یک **زیان کلی (Total loss)** ترکیب می‌شوند، که با فرمول زیر محاسبه می‌شود، تا بین حفظ اطلاعات ورودی (عملکرد خوب در بازسازی) و سازمان‌یافتگی فضای نهفته تعادل برقرار شود.

$$\mathcal{L}_{VAE} = D_{KL}(\mathcal{N}(\mu, \sigma) || \mathcal{N}(0, I)) + \frac{1}{L} \sum_{l=1}^L \log p(x|z_l; \vartheta)$$

شکل 23: رابطه تابع هزینه نهایی در مقاله

۲-۴. آموزش مدل

در این بخش مدلی را که در بالا تعریف کردیم مطابق با فرآیند توضیح داده‌شده در بخش A.4 مقاله آموزش می‌دهیم.

ابتدا داده‌های پردازش‌شده‌ای را که پیش‌تر ایجاد کرده بودیم بارگذاری کرده و ابرپارامترهایی را که برای آموزش مدل استفاده خواهیم کرد تعریف می‌کنیم.

نکته‌ای که باید به آن توجه داشت این است که مقاله از ۵۰۰۰ اپیاک برای آموزش مدل استفاده کرده، اما به دلیل محدودیت زمانی و سخت‌افزاری، ما از تعداد کمتری اپیک استفاده خواهیم کرد. (بعداً به علت جواب‌های بد مدل با ۵۰۰۰ اپیاک هم امتحان شد:)

• همچنین قصد داشتیم از داده‌های تست و اعتبارسنجی استفاده کنیم، اما چون صورت سؤال گفته بود که فقط باید یک مجموعه داده بسازیم، فقط یک مجموعه‌ی آموزش (train) ساختیم.

برای آموزش مدل، ما تنها از تصاویر نرمال (سالم) استفاده می‌کنیم. ایده‌ی پشت این کار آن است که VAE فقط بر روی تصاویر سالم آموزش ببیند و در بازسازی آن‌ها تخصص پیدا کند. بنابراین زمانی که در بخش‌های

بعدی تصاویر پلپ (ناهنجار) به آن داده شود، بتوان از عملکرد ضعیف آن در بازسازی برای ارزیابی استفاده کرد (به عنوان تشخیص ناهنجاری).

وقتی به مدل تصویر پلپ داده می‌شود، چون دانش محدودی از آن دارد، احتمالاً نسخه‌ای تاری یا مخدوش از تصویر تولید می‌کند و تلاش می‌کند پلپ را شبیه بافت سالم بازسازی کند. خطای بازسازی (reconstruction error) بالا در این حالت، همان چیزی است که برای شناسایی ناهنجاری‌ها استفاده خواهد شد.

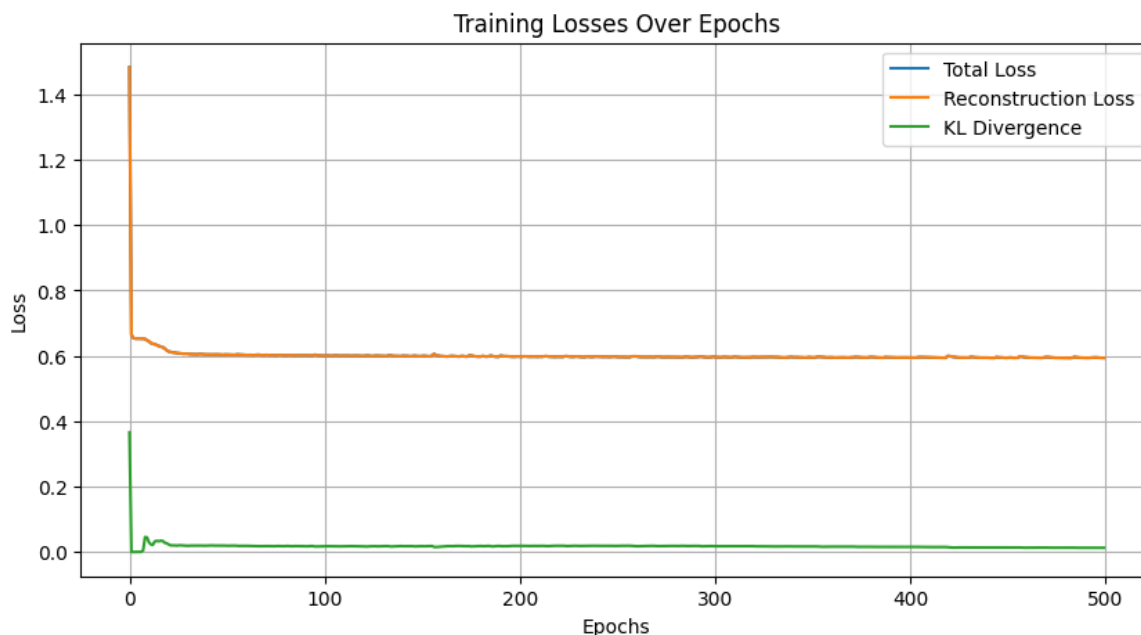
از آنجایی که پس از آموزش متوجه شدیم کیفیت تصاویر تولیدشده پایین است و مقدار زیان KL Divergence نسبت به زیان بازسازی بسیار کم بود، وزن آن را افزایش دادیم تا مدل را بهتر تنظیم کنیم. اما حتی پس از Fine tune کردن و اجرای مدل به مدت ۵۰۰۰ اپک نیز نتایج خیلی خوبی بدست نیامد.



شکل 24: نمودار تابع هزینه مدل در حین ۵۰۰۰ اپاک آموزش!

می‌توان مشاهده کرد که پس از حدود ۵۰۰ اپک، تغییر قابل توجهی در مقدار زیان (Loss) رخ نداد.

همچنین در ادامه، نتیجه یکی دیگر از مدل‌ها که در میان مقادیر مختلف هایپرایامترهای آزمایش‌شده، بهترین نتایج را به‌دست آورده است را هم نشان خواهیم (پارامترهای آن در بخش Test 2 کد قابل مشاهده هستند).



شکل 25: نمودار تابع هزینه مدل دوم در حین ۵۰۰ اپیاک آموزش

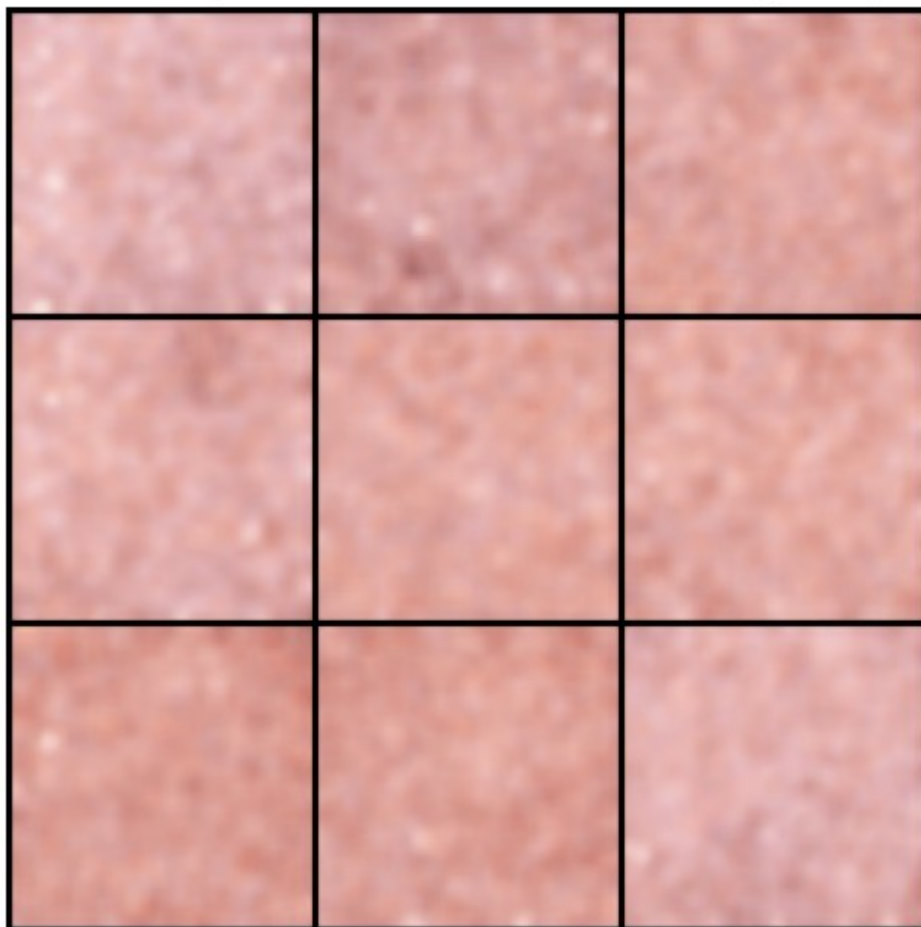
۲-۵. تولید و بازسازی کیفی

در این بخش قصد داریم چند نمونه از تصاویر تولیدشده جدید (از اعداد تصادفی) و همچنین توانایی بازسازی مدل هنگام دریافت تصاویر پلپ دیده‌نشده را بررسی کنیم.

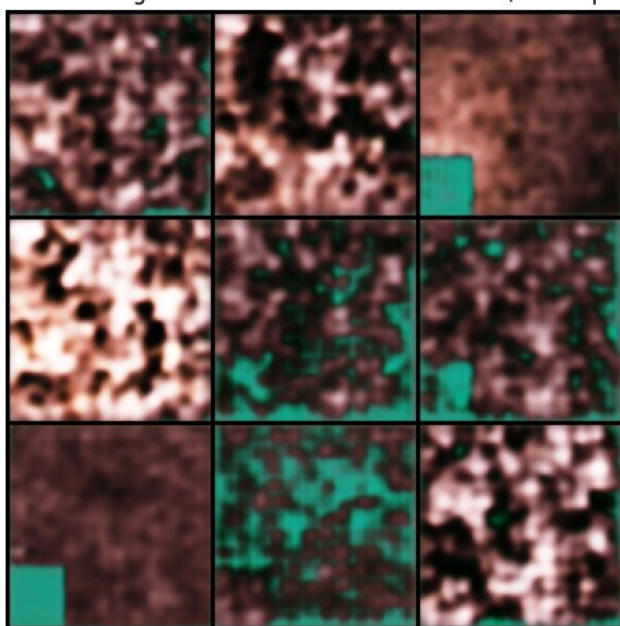
۲-۵-۱. تولید (Generation)

در این قسمت، ۹ عدد تصادفی را از توزیع نرمال استاندارد تولید کرده و آن‌ها را به بخش دیکودر مدل می‌دهیم، سپس تصاویر تولیدشده را در قالب یک جدول 3×3 نمایش می‌دهیم.

Generated Images from Random Latent Vectors (500 Epochs)



Generated Images from Random Latent Vectors (5000 Epochs)

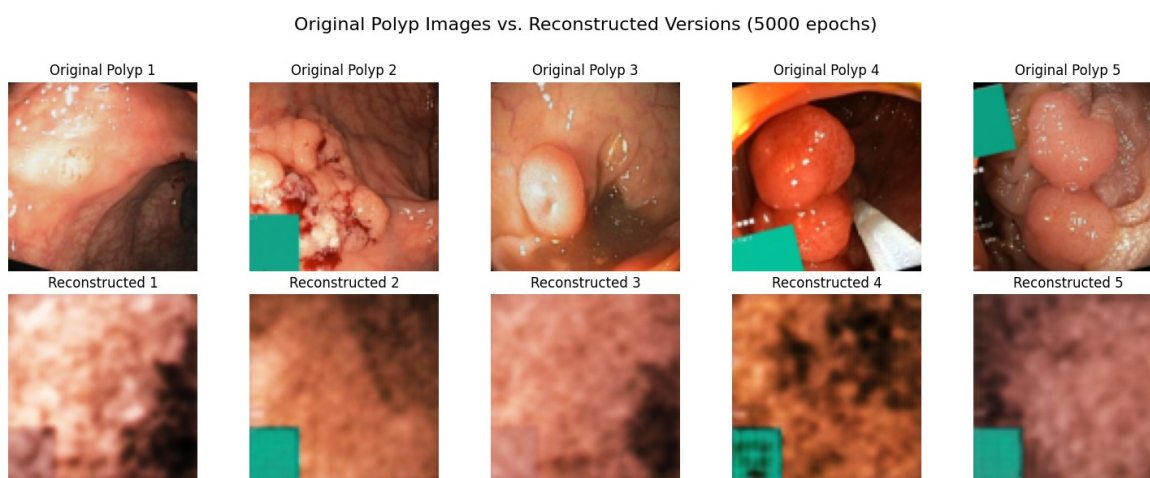


شکل 27: تصاویر تولید شده توسط مدل آموزش دیده با ۵۰۰۰ اپاک

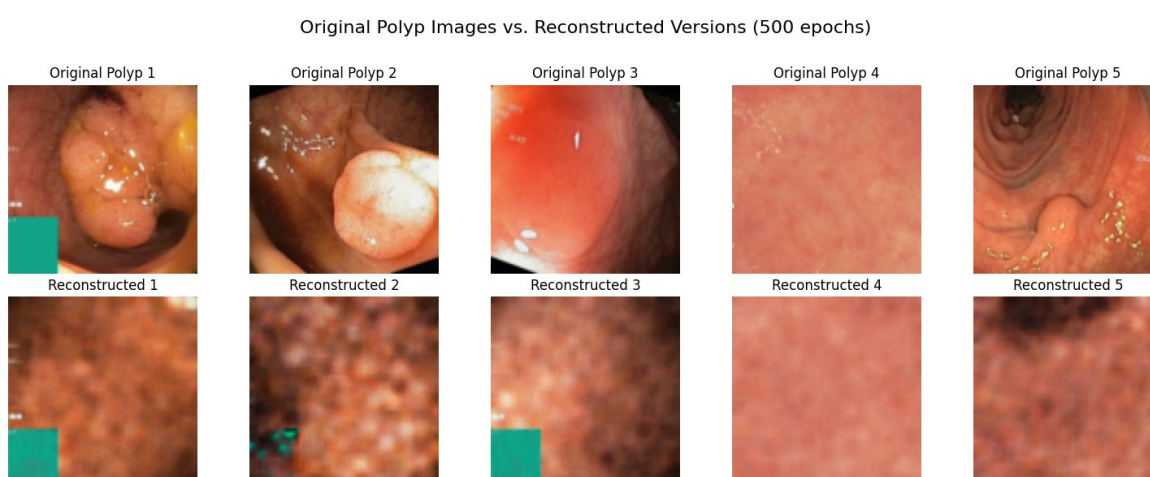
همان‌طور که مشاهده می‌کنیم، با وجود تنظیمات فراوان و اجرای مدل به مدت ۵۰۰۰ اپیاک و انتخاب بهترین مدل ذخیره‌شده، عملکرد مدل در تولید تصاویر همچنان نسبتاً ضعیف بود.

۲-۵-۲. بازسازی (Reconstruction)

در این مرحله، چند تصویر شامل پلیپ را به مدل می‌دهیم تا توانایی بازسازی آن را ارزیابی کنیم.



شکل 28: تصاویر بازسازی شده توسط مدلی که ۵۰۰۰ اپیاک آموزش دیده



شکل 29: تصاویر بازسازی شده مدلی که ۵۰۰ اپیاک آموزش دیده

همان‌طور که مشاهده می‌شود، توانایی بازسازی مدل چندان مطلوب نیست، با وجود آموزش گسترده و فیلترگذاری. اما این موضوع تا حدودی قابل انتظار است، چرا که مدل هرگز تصاویر پلیپ را در طول آموزش ندیده است.

۲-۵. ارزیابی عددی (Numeric Evaluation)

در این بخش، ۵۰ تصویر از پوشه processed/polyp را بازسازی کرده و عملکرد بازسازی آن‌ها را با استفاده از معیارهای PSNR و SSIM ارزیابی می‌کنیم.

اما ابتدا این معیارها را به‌طور خلاصه توضیح می‌دهیم:

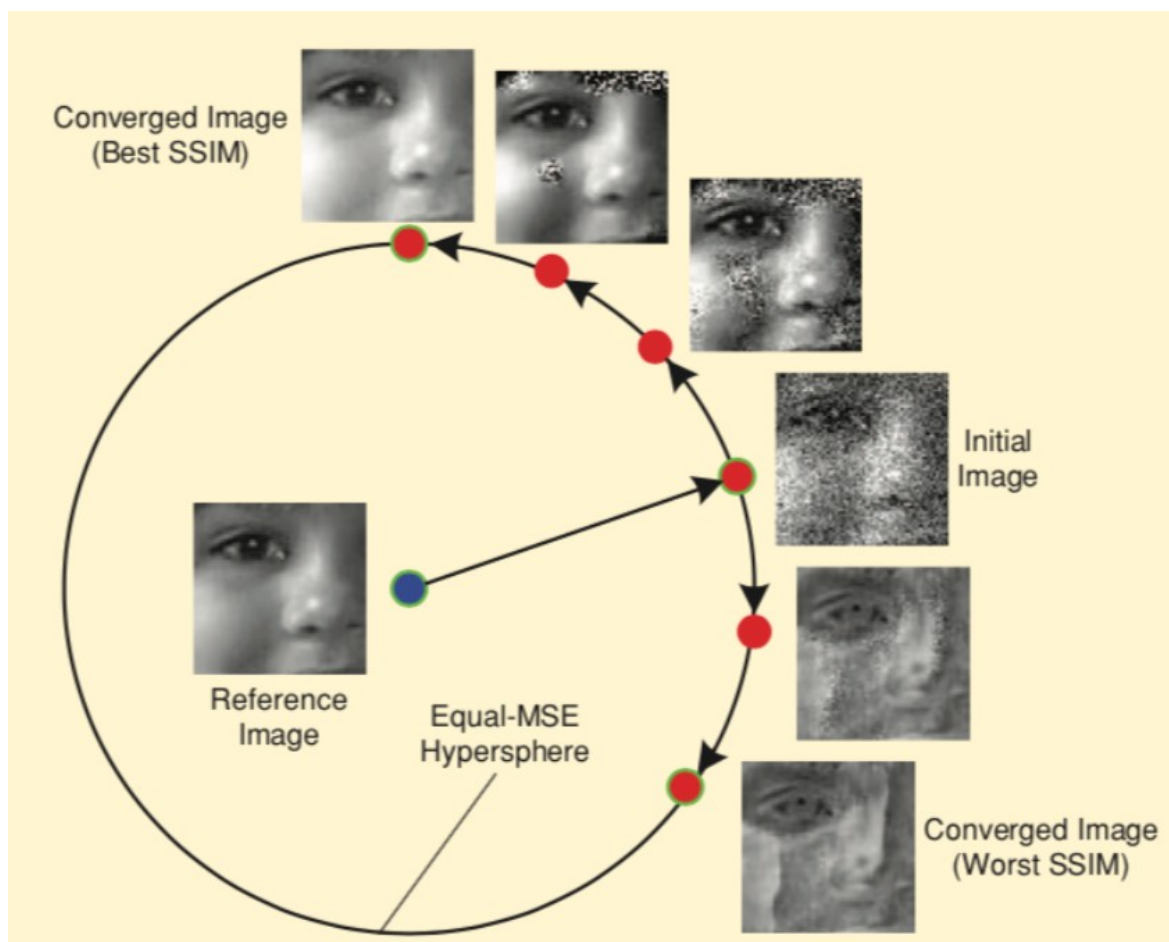
PSNR (Peak Signal to Noise Ratio) کیفیت یک تصویر را با مقایسه‌ی بیشینه مقدار پیکسل با میزان نویز (یا خطا) اندازه‌گیری می‌کند. این مقدار بر حسب دسی‌بل (dB) بیان می‌شود و هر چه مقدار PSNR بیشتر باشد، کیفیت تصویر (بازسازی) بالاتر بوده و شباهت بیشتری با تصویر اصلی دارد. (محاسبات به صورت پیکسل به پیکسل انجام می‌شود!)

$$PSNR = 10 \cdot \log_{10} \left(\frac{MAX^2}{MSE} \right)$$

 FastPix

شکل 30: رابطه PSNR

SSIM (Structural Similarity Index Measure) یک معیار پیشرفته‌تر است که شباهت بین دو تصویر را می‌سنجد. برخلاف PSNR که فقط به خطاهای پیکسل به پیکسل نگاه می‌کند، SSIM تغییرات در بافت، روشنایی و کنتراست را نیز در نظر می‌گیرد. مقدار SSIM بین -۱ تا ۱ است. عددی نزدیک به ۱ نشان‌دهنده‌ی شباهت ساختاری بسیار بالا بین دو تصویر است.



شکل 31: توضیح SSIM

PSNR (dB)	[Mean $\pm$ Std Dev]	SSIM	[Mean $\pm$ Std Dev]
16.23	± 2.72	0.3798	± 0.1106

شکل 32: حاصل معیار های PSNR و SSIM برای مدلی که در ۵۰۰۰ اپیاک آموزش دیده

PSNR (dB)	[Mean $\pm$ Std Dev]	SSIM	[Mean $\pm$ Std Dev]
16.30	± 2.33	0.3966	± 0.0831

شکل 33: حاصل معیار های PSNR و SSIM برای مدلی که در ۵۰۰ اپیاک آموزش دیده

در اینجا می بینیم که هر دو مدل آزمایش شده با وجود تفاوت های بصری، مقادیر بسیار نزدیکی از PSNR و SSIM را به دست آوردند.

۲-۷. تحلیل نتایج نهایی و بحث

در این بخش به چند سؤال در مورد نتایج به دست آمده پاسخ خواهیم داد.

۲-۷-۱. تحلیل کیفی

۲-۷-۱-۱. تصاویر تولید شده تا چه حد واقع گرایانه اند؟

متأسفانه، با وجود تمام تلاش‌ها و صرف زمان برای Fine tune کردن مدل و حتی اجرای آن به مدت ۵۰۰۰ اپیاک همان‌طور که در مقاله ذکر شده بود، نتایج به دست آمده رضایت‌بخش نبودند و در عمل این نتایج کاربردی در شرایط واقعی نخواهند داشت. مدل تا حدی توانست رنگ صورتی و برخی لکه‌های سیاه را بازسازی کند، اما به‌طور کلی تصاویر بسیار تار بودند و بخش سبز موجود در برخی از تصاویر ورودی (احتمالاً یک Overlay هنگام عکس برداری بودند) به صورت برجسته‌ای در تصاویر تولیدی ظاهر شده بود.

۲-۷-۱-۲. بازسازی پلپ چه جزئیاتی را حفظ کرده و چه بخش‌هایی محو شده اند؟

در شرایط ایده‌آل (مدلی که بتواند نتایج بهتری نسبت به آنچه داریم تولید کند)، به احتمال زیاد رنگ، نورپردازی، پس‌زمینه و بافت نواحی سالم تصویر حفظ خواهند شد، اما بازسازی خود پلپ با شکست مواجه می‌شود، چراکه مدل آن را در طول آموزش ندیده است. بنابراین، بخش‌هایی از تصویر که پلپ در آن‌ها قرار دارد احتمالاً تار، محو و با بافتی متفاوت خواهند بود.

۲-۷-۲. تحلیل کمی

۲-۷-۲-۱. مقادیر PSNR و SSIM چه نکاتی درباره کیفیت بازسازی نشان می‌دهند؟

این سؤال در قسمت‌های قبل پاسخ داده شده و در اینجا همان متن کپی شده است:

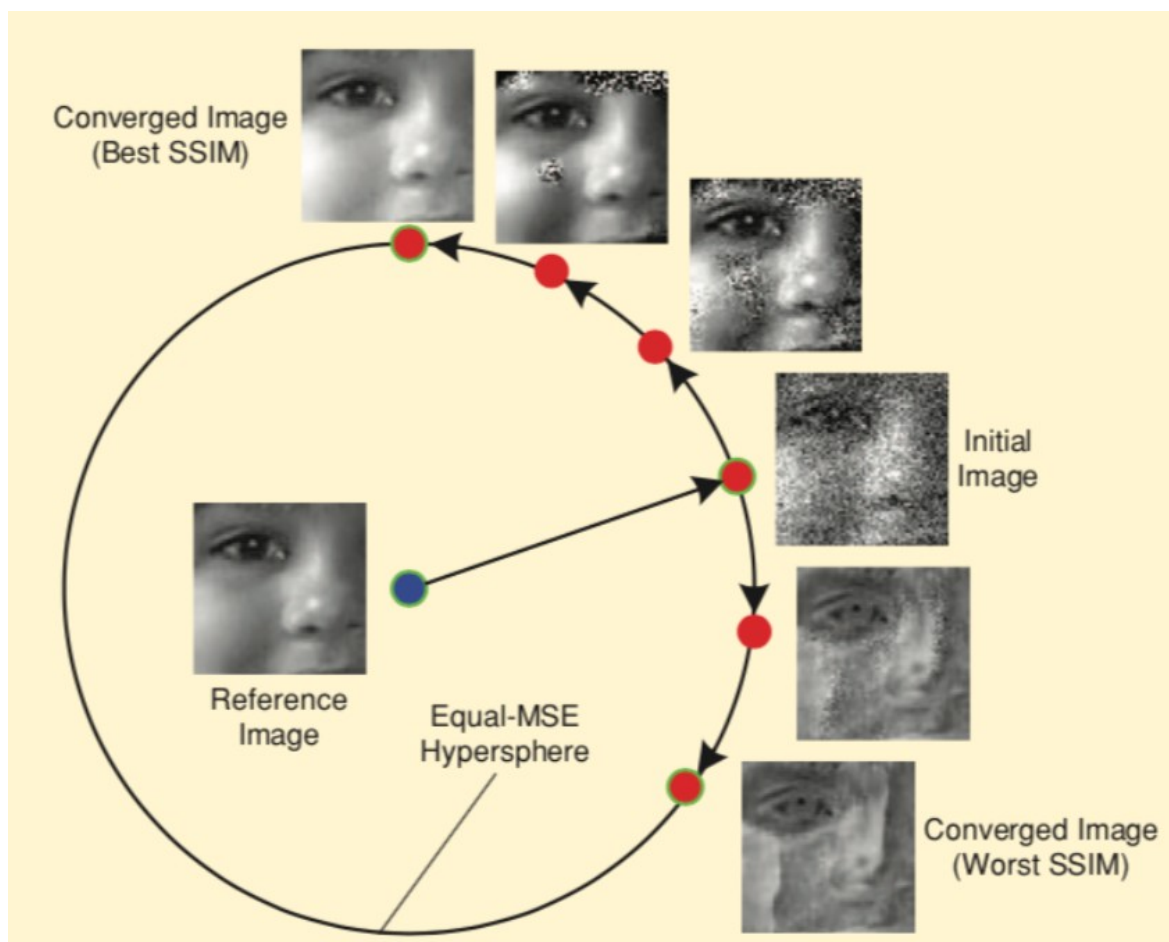
PSNR (Peak Signal to Noise Ratio) کیفیت یک تصویر را با مقایسه‌ی بیشینه مقدار پیکسل با میزان نویز (یا خطا) اندازه‌گیری می‌کند. این مقدار بر حسب دسی‌بل (dB) بیان می‌شود و هر چه مقدار PSNR بیشتر باشد، کیفیت تصویر (بازسازی) بالاتر بوده و شباهت بیشتری با تصویر اصلی دارد. (محاسبات به صورت پیکسل به پیکسل انجام می‌شود!)

$$PSNR = 10 \cdot \log_{10} \left(\frac{MAX^2}{MSE} \right)$$

 FastPix

شکل 34: رابطه PSNR

SSIM (Structural Similarity Index Measure) یک معیار پیشرفته‌تر است که شباهت بین دو تصویر را می‌سنجد. برخلاف PSNR که فقط به خطاهای پیکسل به پیکسل نگاه می‌کند، SSIM تغییرات در بافت، روشنایی و کنتراست را نیز در نظر می‌گیرد. مقدار SSIM بین -۱ تا ۱ است. عددی نزدیک به ۱ نشان‌دهنده شباهت ساختاری بسیار بالا بین دو تصویر است.



شکل 35: توضیح SSIM

۲-۷-۲-۲. اگر مقادیر PSNR و SSIM پایین هستند، احتمالاً مشکل در کدام بخش (معماری، هایپرایامتر یا پیش پردازش) است؟

با توجه به اینکه ما از دیتاست متفاوتی نسبت به مقاله استفاده کرده ایم و مدل تصاویر پلیپ را در طول آموزش ندیده است، تا حدی مقادیر پایین PSNR و SSIM قابل انتظار هستند.

اما برای بهبود عملکرد مدل، می توان جنبه های زیر را تغییر داد (با توجه به اینکه تنظیمات هایپرایامترها به صورت گسترده انجام شده، احتمال اینکه مشکل اصلی از آن باشد پایین است):

داده های آموزش: واضح ترین وجه تفاوت، استفاده از دیتاست متفاوت با

مقاله ی اصلی است. علاوه بر این، مدل روی تصاویر پلیپ آموزش ندیده است، بنابراین برای بهبود بازسازی پلیپ، باید این تصاویر را نیز در مجموعه آموزش قرار دهیم. همچنین استفاده از داده های آموزشی بیشتر یا تکنیک های افزایش داده (data augmentation) شدیدتر می تواند مفید باشد.

وزن‌دهی به تابع زیان: تعادل میان زیان بازسازی و واگرایی KL اهمیت بالایی دارد. افزایش وزن KL منجر به ایجاد فضای نهفته ساختارمندتری می‌شود، اما کیفیت بازسازی را کاهش داده و موجب کاهش PSNR/SSIM می‌شود. از طرفی کاهش وزن KL می‌تواند بازسازی را بهبود بخشد، اما ممکن است به کیفیت تولید (generation) ضربه بزند. البته در طول آموزش ما این مورد را آزمایش کردیم، اما متأسفانه مفید واقع نشد، چراکه تابع هزینه KL به سرعت به صفر می‌رسید.

ظرفیت مدل: استفاده از معماری VAE عمیق‌تر یا پهن‌تر می‌تواند باعث شود مدل جزئیات بیشتری را یاد بگیرد و بازسازی باکیفیت‌تری انجام دهد و مقادیر بهتری از PSNR/SSIM کسب کند.

ابعاد فضای نهفته (Latent Dimension): افزایش latent_dim می‌تواند به مدل امکان دهد اطلاعات بیشتری درباره تصاویر ذخیره کند، که ممکن است باعث بهبود کیفیت بازسازی شود، ولی در عوض سازمان‌دهی فضای نهفته سخت‌تر خواهد شد.

۲-۸. بخش امتیازی

در این بخش، یک شبکه عصبی CNN ساده تعریف کرده و آن را برای تشخیص تفاوت بین تصاویر واقعی پلپ و تصاویر بازسازی‌شده از پلپ آموزش می‌دهیم. با توجه به کیفیت پایین بازسازی‌ها، انتظار داریم که این مدل بتواند با دقتی نزدیک به ۱۰۰٪ این دو را از یکدیگر تشخیص دهد.

۲-۸-۱. ساخت دیتاست

در ابتدا، دیتاست مورد نظر را ایجاد کرده و آن را به دو بخش تقسیم می‌کنیم: ۸۰٪ برای آموزش و ۲۰٪ برای تست.

۲-۸-۲. ساخت و آموزش CNN

سپس یک طبقه‌بند ساده مبتنی بر CNN تعریف کرده و آن را آموزش می‌دهیم.

```
Epoch [1/15], Loss: 0.6867
Epoch [2/15], Loss: 0.6626
Epoch [3/15], Loss: 0.5849
Epoch [4/15], Loss: 0.4908
Epoch [5/15], Loss: 0.2643
Epoch [6/15], Loss: 0.4192
Epoch [7/15], Loss: 0.3024
Epoch [8/15], Loss: 0.2266
Epoch [9/15], Loss: 0.1216
Epoch [10/15], Loss: 0.1473
Epoch [11/15], Loss: 0.0768
Epoch [12/15], Loss: 0.2256
Epoch [13/15], Loss: 0.2888
Epoch [14/15], Loss: 0.1813
Epoch [15/15], Loss: 0.1999
Classifier training complete.
```

شکل 36: روند آموزش CNN

۲-۸-۳. ارزیابی شبکه‌ی CNN

در نهایت، مدل را با محاسبه دقت (Accuracy) و مساحت زیر منحنی ROC (AUC) ارزیابی می‌کنیم.

```
Accuracy: 93.75%
AUC: 0.9925
```

شکل 37: دقت مدل CNN

همان‌طور که مشاهده می‌شود، دقت مدل ۹۳٪ و مقدار AUC نزدیک به ۱ است، که نشان می‌دهد توانایی بازسازی مدل به هیچ وجه آن قدر قوی نبوده که بتواند طبقه‌بند را فریب دهد. بنابراین، تصاویر بازسازی شده نمی‌توانند جایگزینی مناسب برای دیتاست واقعی باشند.

